

Questão 1:

(A) R: O único caminho viável é o pós-ordem. Os outros dois, destroem primeiro o pai ao invés do filho, causando vazamento de memória.

(B) R: Com ponteiro simples, depois do `free()` o ponteiro na main continuaria apontando para um endereço que já foi liberado. Com o ponteiro duplo, depois do `free()` fazemos `*ppRaiz = NULL`, o que faz o ponteiro original na main virar `NULL` automaticamente. Assim não tem como acessar memória inválida por acidente.

Questão 2:

R: Numa AVL a altura máxima é sempre $O(\log n)$, então a recursão empilha no máximo $O(\log n)$ frames na pilha. Numa BST comum sem balanceamento, se os dados forem inseridos em ordem crescente a árvore vira uma lista, a altura chega a $O(n)$ e a recursão empilha $O(n)$ frames. Para grandes volumes de dados isso causa estouro de pilha (stack overflow) e derruba o programa.

Questão 3:

R: O `fb` é um `int` de 32 bits mas só usa 3 valores: -1, 0 e 1. Para representar 3 valores bastam 2 bits. Em sistemas embarcados dá pra usar bit-fields do C, declarando `signed int fb : 2` no lugar de `int fb`, reduzindo o campo de 32 bits para 2 bits por nó, economizando memória em cada nó da árvore.